Formal Verification

Aave V4 Tokenization Spoke

April 2026

Prepared for Aave Labs

Table of Contents

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Platform
Aave V4 Tokenization Spoke	https://github.com/aave/aave-v4/tree/v0.5.9	c3b9287	Solidity

Project Overview

This document describes the specification and verification of **AAVE V4 Tokenization Spoke**. The work was undertaken from **March 11th, 2026** to **April 13th, 2026**.

The following contract list is included in our scope:

`v0.5.9/src/spoke/TokenizationSpoke.sol`
`v0.5.9/src/spoke/instances/TokenizationSpokeInstance.sol`

The Certora Prover demonstrated that the implementation of the Solidity contracts of the Spoke is correct with respect to the formal rules written by the Certora team.

No issues discovered during the verification process.

Protocol Overview

TokenizationSpoke is an ERC4626-compliant vault that tokenizes Hub addedShares positions, allowing users to deposit underlying assets into the Hub and receive transferable ERC20 share tokens representing their added liquidity.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	-	-	-
High	-	-	-
Medium	-	-	-
Low	-	-	-
Informational	1	1	-
Total	**1**	**1**	**-**

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
		Likelihood		

Informational Issues

I-01. Redeemed shares are at most maxRedeem(owner)

Description: The ERC4626 specification requires that redeem(maxRedeem(owner)) must never revert for a valid owner. This invariant is violated in TokenizationSpoke. When computing maxRedeem, available liquidity retrieved from the hub is rounded down during conversion to shares, causing maxRedeem to underestimate the actual redeemable amount. Formal verification confirms the discrepancy is at most 1 share (rule redeemRespectsMaxRedeem).

Recommendations: Document the rounding behavior explicitly in the natspec of maxRedeem. Rounding up is not a safe fix as it may overestimate available liquidity and cause reverts for a different reason. Callers should be made aware that maxRedeem may be off by 1 share.

Customer's response: Acknowledged, will add natspec documentation clarifying the rounding behavior.

Formal Verification

Verification Methodology

We performed verification of the **AAVE V4** protocol using the Certora verification tool which is based on Satisfiability Modulo Theories (SMT). In short, the Certora verification tool works by compiling formal specifications written in the Certora Verification Language (CVL) and **AAVE's** implementation source code written in Solidity.
More information about Certora's tooling can be found in the Certora Technology Whitepaper.

If a property is verified with this methodology it means the specification in CVL holds for all possible inputs. However specifications must introduce assumptions to rule out situations which are impossible in realistic scenarios (e.g. to specify the valid range for an input parameter). Additionally, SMT-based verification is notoriously computationally difficult. As a result, we introduce overapproximations (replacing real computations with broader ranges of values) and underapproximations (replacing real computations with fewer values) to make verification feasible.

Rules: A rule is a verification task possibly containing assumptions, calls to the relevant functionality that is symbolically executed and assertions that are verified on any resulting states from the computation.

Inductive Invariants: Inductive invariants are proved by induction on the structure of a smart contract. We use constructors as a base case, and consider all other (relevant) externally callable functions that can change the storage as step cases.
Specifically, to prove the base case, we show that a property holds in any resulting state after a symbolic call to the respective constructor. For proving step cases, we generally assume a state where the invariant holds (induction hypothesis), symbolically execute the functionality under investigation, and prove that after this computation any resulting state satisfies the invariant.

Verification Notations

Formally Verified	The rule is verified for every state of the contract(s), under the assumptions of the scope/requirements in the rule.
Acknowledged Violation	In case of a violation diagnosed as informational

General Assumptions and Simplifications

- We use our own implementation of `Math.mulDiv, Math.mulDivUp` and `Math.mulDivDown.` These are equivalent to the originals and better suited for verification purposes.
- Loops are inherently difficult for formal verification. We handle loops by unrolling them a specific amount of times. We use a **loop_iter of 3**, unrolling each loop up to three twice.
- Underlying assets are assumed to be standard ERC20, specifically:
    a. No fees on transfer
    b. Without double entries
    c. Decimals between 6 and 18
- The verified Solidity contracts are compiled with solc8.28.
- All properties are verified against the provided Hub.sol implementation.
- Front-running safety properties and rule dustFavorsTheHouse are verified with a simplified symbolic representation of the Hub

Properties Overview

ID	Title	Impact
P-01	ERC4626 compliance	Direct fund loss, broken economic model, potential insolvency
P-02	TokenizationSpoke - Hub integrity	Broken integrity between Hub and Spoke
P-03	TokenizationSpoke bounds integrity	ERC4626 compliance with Hub integrity
P-04	TokenizationSpoke front running safety	Denial of service
P-05	TokenizationSpoke integrity	Integrity of core functionality
P-06	TokenizationSpoke valid state	Solvency and broken data model

P-01. ERC4626 compliance

Status: Formally Verified

Rule Name	Status	Description	Link to rule report

totalAssetsIsAtLeastAsMuchAsTotalSupplyOfShares	Verified	*ERC4626-style solvency: totalAssets is at least totalSupply in share units and zeros align*	rule report
convertToSharesAndAssetsInverse	Verified	*Verifies ERC4626 requirement: convertToAssets(convertToShares(assets)) == assets (within rounding)*	rule report
convertToAssetsAndSharesInverse	Verified	*Verifies ERC4626 requirement: convertToShares(convertToAssets(shares)) == shares (within rounding)*	rule report
previewDepositMatchesConvertToShares	Verified	*Verifies that previewDeposit returns the same value as convertToShares*	rule report
previewMintMatchesConvertToAssets	Verified	*Verifies that previewMint returns the same value as convertToAssets within rounding*	rule report
previewWithdrawMatchesConvertToShares	Verified	*Verifies that previewWithdraw returns the same value as convertToShares within rounding*	rule report
previewRedeemMatchesConvertToAssets	Verified	*Verifies that previewRedeem returns the same value as convertToAssets*	rule report
totalAssets_equalsHubBalance	Verified	*totalAssets matches previewRedeem(totalSupply()) for the live exchange rate*	rule report
convertToAssetsWeakAdditivity	Verified	*For share amounts that sum safely, converting each chunk to assets and adding never exceeds converting the combined shares once (weak superadditivity of the rate).*	rule report
convertToSharesWeakAdditivity	Verified	*For asset amounts that sum safely, converting each chunk to shares and adding never exceeds converting the combined assets once.*	rule report

conversionWeakMonotonicity	Verified	*Larger share amounts map to at least as many assets; larger asset amounts map to at least as many shares (weak monotonicity of convertToAssets and convertToShares).*	rule report
conversionWeakIntegrity	Verified	*convertToShares(convertToAssets(x)) never exceeds x in shares; convertToAssets(convertToShares(x)) never exceeds x in assets.*	rule report
convertToCorrectness	Verified	*Assets are at least convertToAssets(convertToShares(assets)); shares are at least convertToShares(convertToAssets(shares))—round-trips never inflate the original quantity.*	rule report

P-02. TokenizationSpoke - Hub integrity

Status: Formally Verified

Rule Name	Status	Description	Link to rule report
totalSupplyHubSupplySharesIntegrity	Verified	*Invariant: ERC20 totalSupply() equals hub.getSpokeAddedShares(ASSET_ID, currentContract)—on-chain accounting matches the hub's spoke share ledger.*	rule report

P-03. TokenizationSpoke bounds integrity

Status: Acknowledged violation

Rule Name	Status	Description	Link to rule report

withdrawRespectsMaxWithdraw	Verified	*After withdraw(assets, receiver, owner), the withdrawn asset amount does not exceed maxWithdraw(e, owner) evaluated under the same environment.*	*rule report*
depositRespectsMaxDeposit	Verified	*After deposit(assets, receiver), the deposited asset amount does not exceed maxDeposit(e, receiver) evaluated under the same environment.*	*rule report*
redeemRespectsMaxRedeem	Violated	*Redeemed shares are at most maxRedeem(owner) plus one share (rounding slack vs strict ERC4626 bound).*	violated strict version: *rule report* verified version: *rule report*
mintRespectsMaxMint	Verified	*After mint(shares, receiver), the amount of minted shares does not exceed maxMint(e, receiver) evaluated under the same environment.*	*rule report*

P-04. TokenizationSpoke front running safety

Status: Formally Verified

Rule Name	Status	Description	Link to rule report
noFrontRunningOnRedeem	Verified	*Swapping order of two redeems from same initial state does not make the first revert*	*rule report*
noFrontRunningOnWithdraw	Verified	*Swapping order of two withdraws from same initial state does not make the first revert*	*rule report*

P-05. TokenizationSpoke integrity

Status: Formally Verified

Rule Name	Status	Description	Link to rule report
deposit_integrity	Verified	*Verifies that deposit operation increases the receiver's share balance and transfers assets and only within the max deposit limit*	*rule report*
zeroDepositZeroShares	Verified	*For any deposit call, the minted share amount is zero if and only if the deposited asset amount is zero.*	*rule report*
mint_integrity	Verified	*Verifies that mint operation increases the receiver's share balance and transfers assets*	*rule report*
withdraw_integrity	Verified	*Verifies that withdraw operation decreases the owner's share balance and transfers assets*	*rule report*
redeem_integrity	Verified	*Verifies that redeem operation decreases the owner's share balance and transfers assets*	*rule report*
redeemingAllValidity	Verified	*If the redeem amount equals the owner's full balanceOf(owner), after redeem the owner's share balance is zero.*	*rule report*
onlyIncreaseOtherUsersShares	Verified	*Third parties (not msg.sender, not hub) never lose shares or asset balance from deposit/mint/withdraw/redeem/transferFrom paths*	*rule report*
maxWithdraw_respectsLiquidity	Verified	*maxWithdraw and maxRedeem are bounded by previewRedeem(balance) and balance respectively*	*rule report*
assetIsThisContract	Verified	*Configuration edge case: no share inflation when underlying is the spoke itself*	*rule report*
dustFavorsTheHouse	Verified	*totalAssets after deposit+redeem is at least totalAssets before (rounding favors vault)*	*rule report*

P-06. TokenizationSpoke valid state

Status: Formally Verified	

Rule Name	Status	Description	Link to rule report
totalSupplySumOfBalances	Verified	*Invariant: totalSupply() equals the sum over all addresses of per-account share balances (ghost token balance model matches aggregate supply).*	*rule report*

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline and is widely used by developers and security researchers during audits and bug bounties.

Certora also provides architectural design reviews, where researchers analyze system design and trust boundaries early to identify structural risks and reduce complexity before implementation.

Certora conducts off-chain audits covering backend services, APIs, frontend, and mobile applications, focusing on vulnerabilities in authentication, data handling, key management, and wallet-related workflows.

Certora also offers Penetration Testing and Red Teaming to simulate real-world attacks and uncover exploitable weaknesses across infrastructure, applications, and business logic.

In addition, Certora provides operational security (OpSec) assessments, reviewing key management, access controls, and operational processes to strengthen overall security posture.

Finally, Certora delivers ongoing monitoring and incident response, enabling continuous threat detection and prevention, alert triage, and coordinated response to active incidents.